

Exception Handling in Java

BICTE



What is an Exception?

An **exception** is an **abnormal condition or event** that occurs **during the execution of a program** and disrupts the normal flow of instructions.

Characteristics of Exception

- Occurs **at runtime**
- Represented as an **object**
- Can be **handled** using exception handling mechanism
- Derived from the Throwable class

Without Exception handling

```
public class ExHandling {  
    Run main | Debug main | Run | Debug  
    public static void main(String args[]) {  
  
        System.out.println(x: "Hello World!");  
        int data = 100/0;  
  
        /* This code is blocked if exception handling is not used */  
        System.out.println(x: "Hello World 2!");  
        System.out.println(x: "Hello World 3!");  
    }  
}
```

With Exception handling

```
1 public class ExHandling {  
    Run main | Debug main | Run | Debug  
2 public static void main(String args[]) {  
3  
4     try{  
5         int data =100/0 ;  
6     }  
7     catch(ArithmeticException ex){  
8         System.out.println(ex);  
9         //makes easy to debug in application .  
10    }  
11    /* This code is blocked if exception handling is not used */  
12    System.out.println(x: "Hello World 2!");  
13    System.out.println(x: "Hello World 3!");  
14 }  
15  
16
```

PROBLEMS 10

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Filter (e.g. text, !excludeText, t...

Code

[Running] cd "c:\Teaching\Java\MRCJava\Code\Chapter3\" && javac ExHandling.java && java ExHandling

java.lang.ArithmeticException: / by zero

Hello World 2!

Hello World 3!

[Done] exited with code=0 in 1.116 seconds

What is an Error?

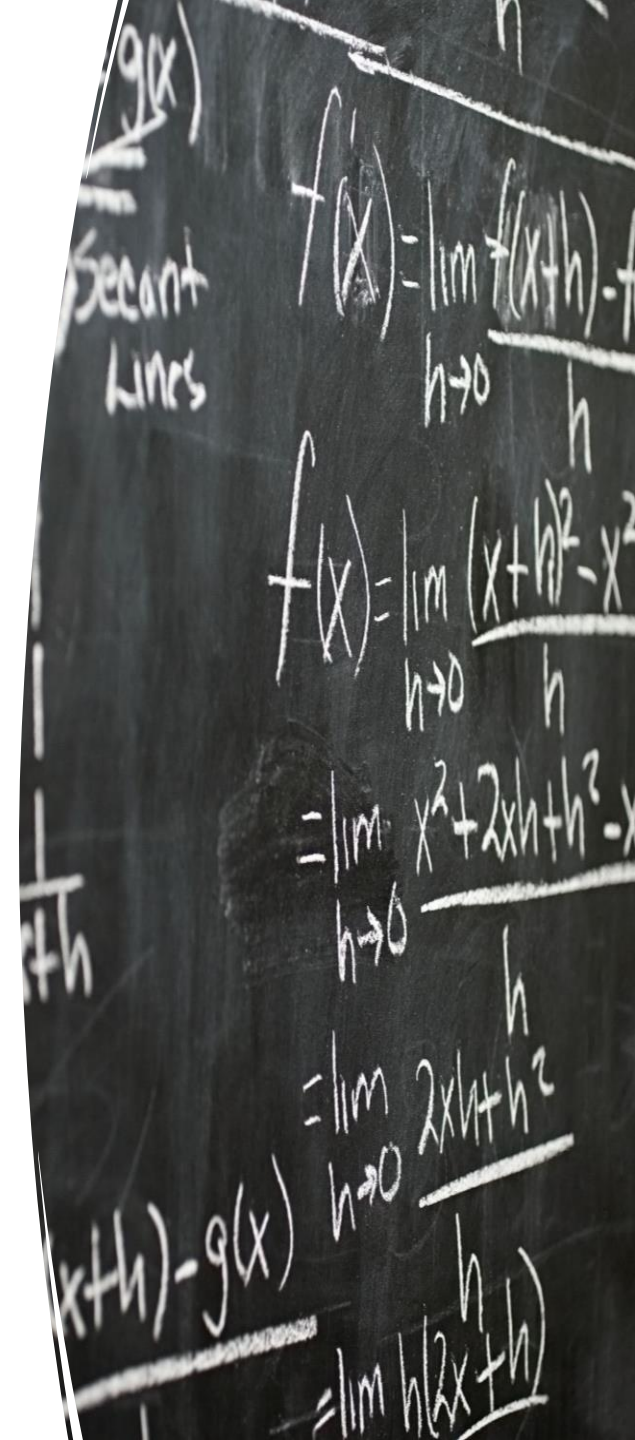
An **error** represents a **serious problem** that occurs due to **system-level failure** and is **beyond the control of the programmer**.

Characteristics of Error

- Occurs due to **JVM or hardware failure**
- Cannot be recovered
- Should **not be handled**
- Program usually terminates

Example of Error

```
public class Test {  
    public static void main(String[] args) {  
        main(args); // infinite recursion  
    }  
}
```



Errors	Exceptions
Errors in java are of type <code>java.lang.Error</code> .	Exceptions in java are of type <code>java.lang.Exception</code> .
All errors in java are unchecked type.	Exceptions include both checked as well as unchecked type.
Errors happen at run time. They will not be known to compiler.	Checked exceptions are known to compiler where as unchecked exceptions are not known to compiler because they occur at run time.
It is impossible to recover from errors.	You can recover from exceptions by handling them through try-catch blocks.
Errors are mostly caused by the environment in which application is running.	Exceptions are mainly caused by the application itself.
Examples : <code>java.lang.StackOverflowError</code> , <code>java.lang.OutOfMemoryError</code>	Examples : Checked Exceptions : <code>SQLException</code> , <code>IOException</code> Unchecked Exceptions : <code>ArrayIndexOutOfBoundsException</code> , <code>ClassCastException</code> , <code>NullPointerException</code>

Need for Exception Handling

Without exception handling:

- Program terminates abruptly
- Remaining code does not execute
- Poor user experience

With exception handling:

- Program continues execution
- Errors are handled gracefully
- Resources are released properly

Keywords and Use in Exception Handling

Keyword	Purpose	Where Used	Key Points	Example
try	Holds risky code	Inside method/block	Must be followed by catch or finally	<code>try { int a = 10/0; }</code>
catch	Handles exception	After try	Executes only if exception occurs	<code>catch (ArithmeticException e) {}</code>
finally	Cleanup code	After try/catch	Always executes (except JVM crash)	<code>finally { close(); }</code>
throw	Throws exception explicitly	Inside method/block	Throws one exception object	<code>throw new Exception("Error");</code>
throws	Declares exception	Method signature	Passes responsibility to caller	<code>void m() throws IOException</code>

Checked Exceptions

Checked exceptions are those exceptions which are **checked by the compiler at compile time**.

The program **will not compile** unless these exceptions are **handled or declared**.

Key Characteristics

- Checked at **compile time**
- Must be handled using **try-catch**
- OR declared using **throws**
- Represent **recoverable conditions**

Examples

- IOException
- SQLException
- ClassNotFoundException
- FileNotFoundException

Unchecked Exceptions

Unchecked exceptions are those exceptions which are **not checked by the compiler** and occur **at runtime**.

Key Characteristics

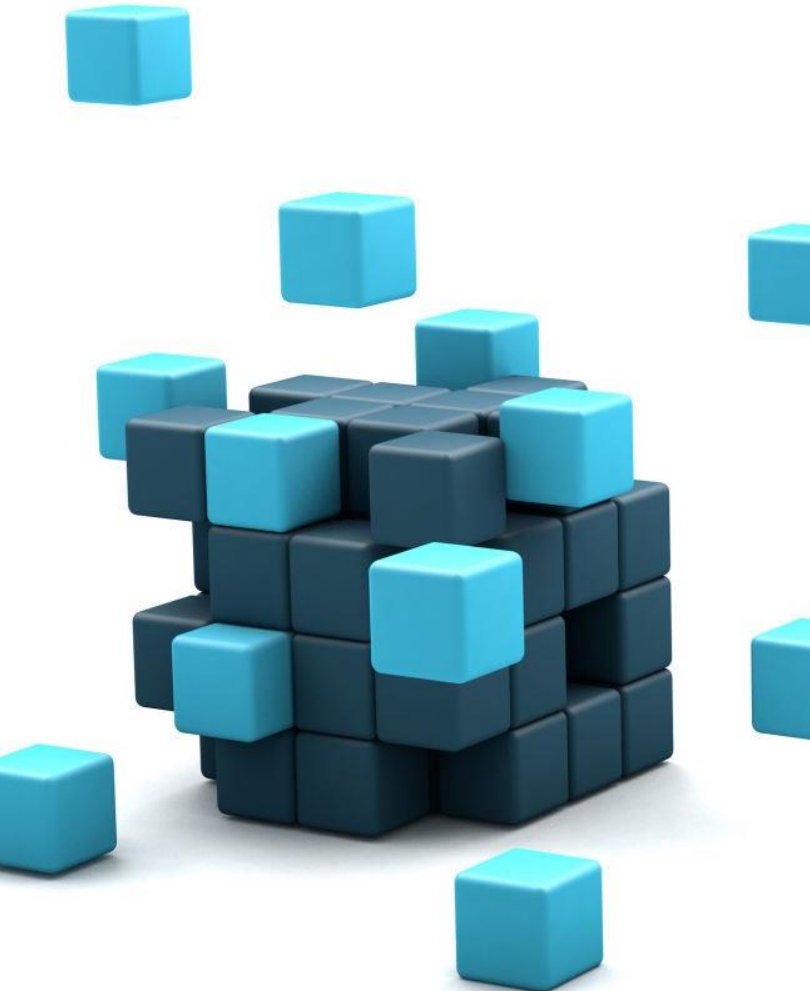
- Checked at **runtime**
- Compiler does **not force handling**
- Usually caused by **programming errors**
- Subclasses of RuntimeException

Examples

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

Checked vs Unchecked Exceptions

Basis	Checked Exceptions	Unchecked Exceptions
Checked by compiler	Yes	No
Occurs at	Compile time	Runtime
Handling mandatory	Yes	No
Package	<code>java.lang.Exception</code> (except <code>RuntimeException</code>)	<code>RuntimeException</code>
Cause	External factors	Programming mistakes
Examples	<code>IOException</code> , <code>SQLException</code>	<code>NullPointerException</code>



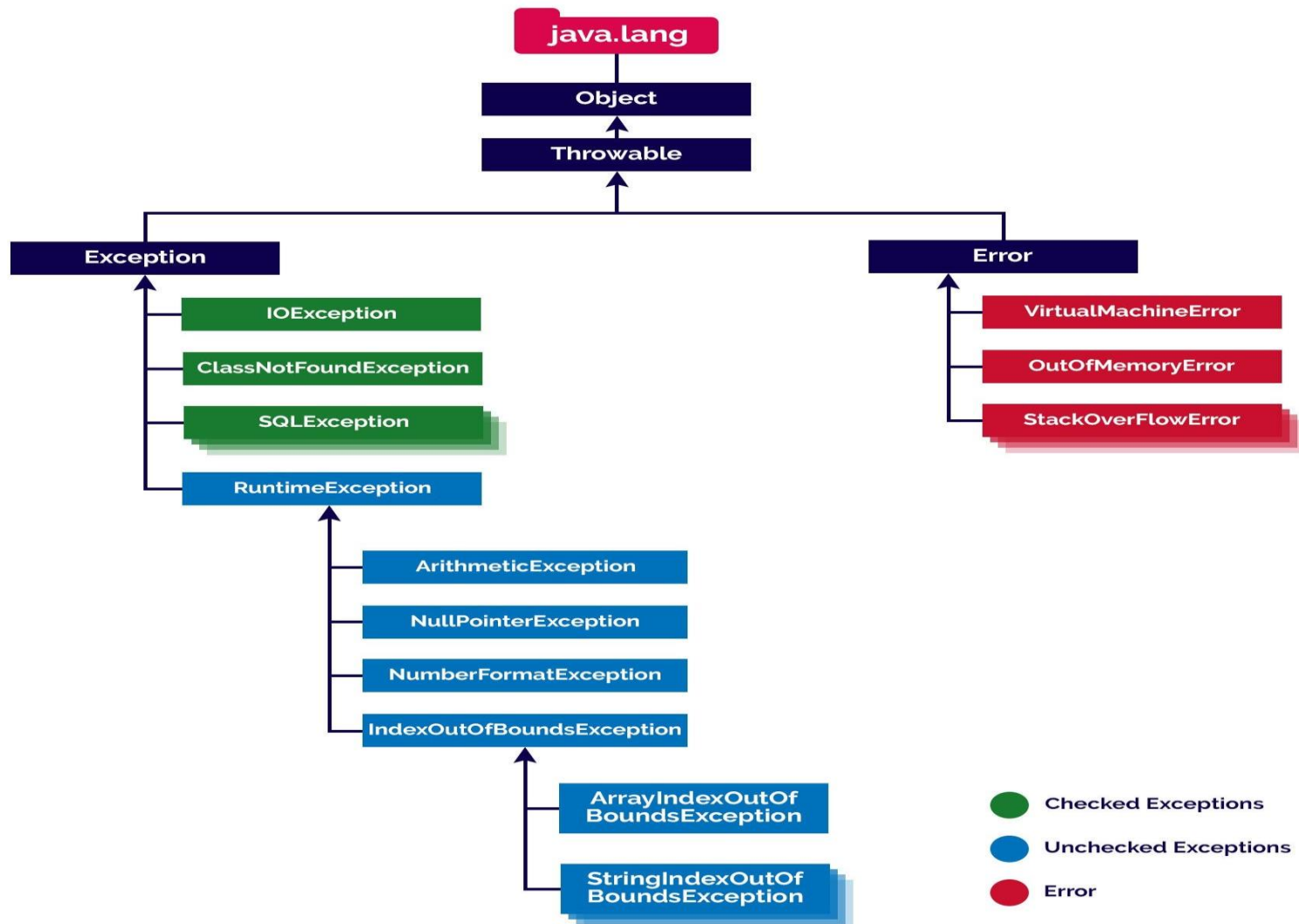
Multiple Exception Handling

- Handling multiple exceptions means **managing more than one possible exception** that may occur in a program using **multiple catch blocks or a single multi-catch block**.
- A single try block may generate **different types of exceptions**, so Java provides mechanisms to handle them **separately or together**.

Multiple Exception Handling

```
class Test {  
    public static void main(String[] args) {  
        try {  
            int[] arr = new int[5];  
            int a = 10 / 0;    // ArithmeticException  
            arr[10] = 50;    // ArrayIndexOutOfBoundsException  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception occurred");  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Exception occurred");  
        }  
        catch (Exception e) {    // General exception  
            System.out.println("General Exception handled");  
        }  
    }  
}
```

Exception Hierarchy



Assignment

Research and write code to handle following exceptions

Most Common Checked Exceptions

- `IOException` – File reading/writing errors
- `FileNotFoundException` – When a file is missing
- `ClassNotFoundException` – Class not found during dynamic loading
- `SQLException` – Database-related errors

Most Common Unchecked (Runtime) Exceptions

- `ArithmeticException` – Division by zero
- `NullPointerException` – Using a null object
- `ArrayIndexOutOfBoundsException` – Invalid array index
- `StringIndexOutOfBoundsException` – Invalid string index
- `NumberFormatException` – Invalid string-to-number conversion
- `IllegalArgumentException` – Invalid method argument